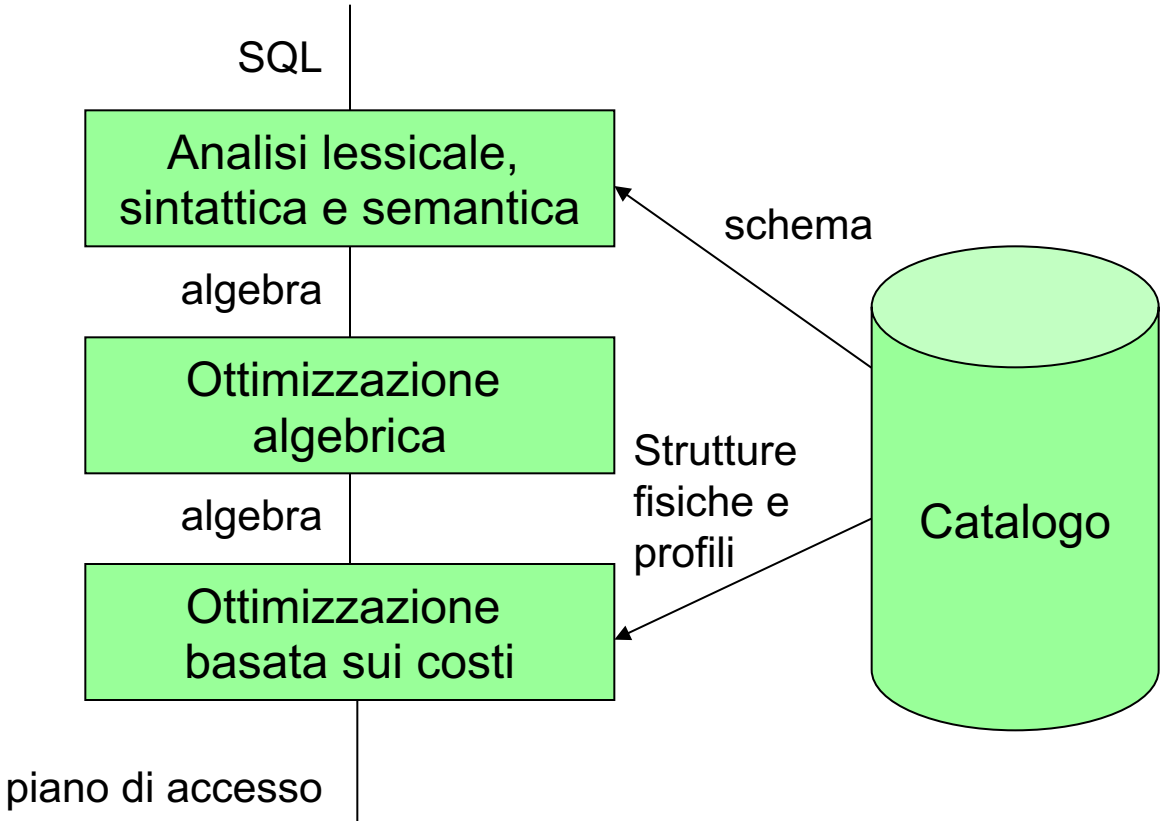
Pipelining vs materializzazione

- Due alternative per le interrogazioni nei sottoalberi:
  - **pipelining**: le ennuple sono "utilizzate" dal nodo superiore a mano a mano che vengono prodotte (anzi, vengono prodotte a richiesta):
    - vantaggio: non dovendo salvare i risultati intermedi, riduce i costi di I/O
    - svantaggio: se i risultati intermedi vengono riutilizzati più volte, è necessario ricalcolarli (ad esempio, il ciclo interno di un nested loop)
  - **materializzazione**: l'intero risultato intermedio viene prodotto e memorizzato, prima di essere utilizzato

Il processo di esecuzione delle interrogazioni



Ottimizzazione basata sui costi

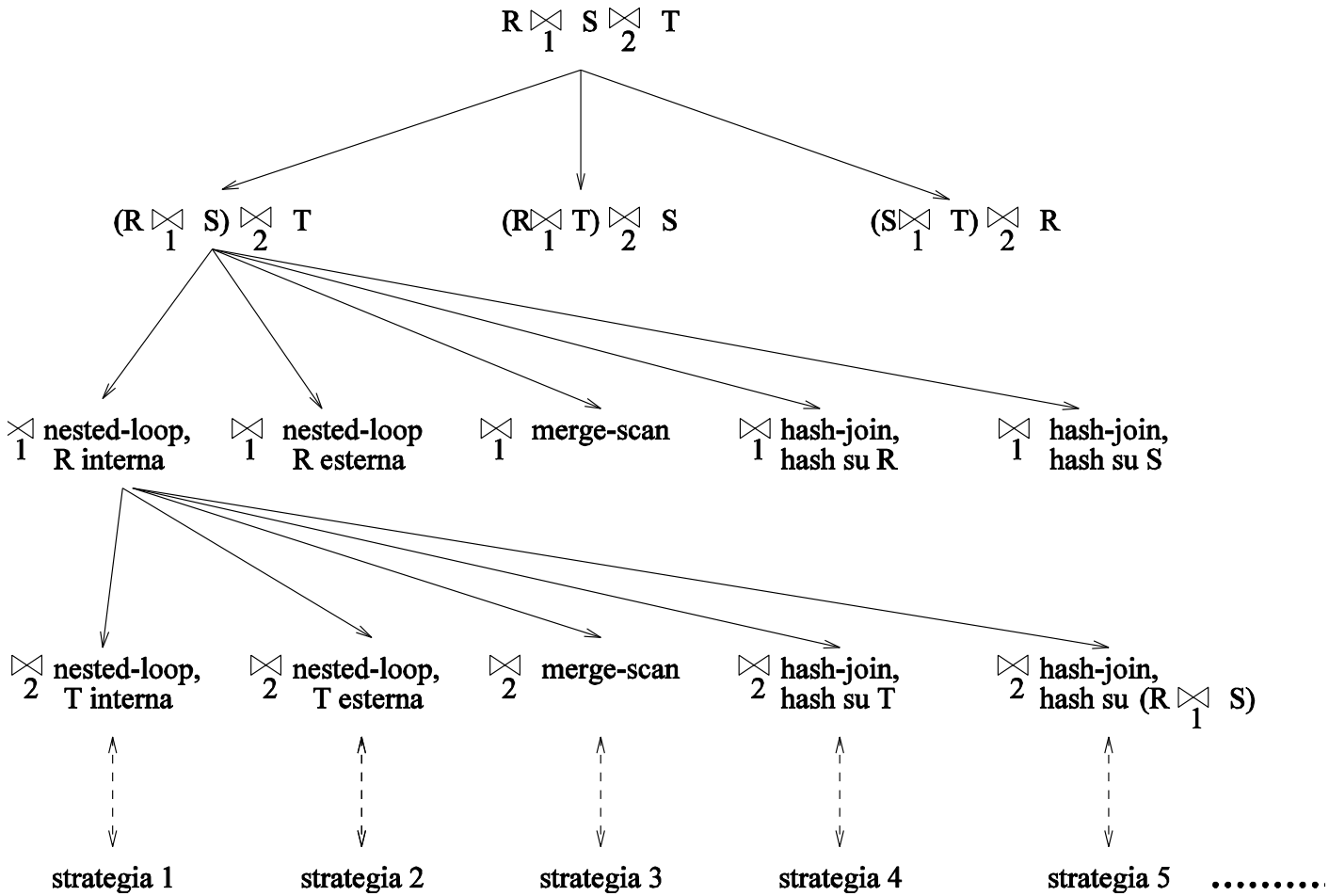
- Un problema articolato, con scelte relative a:
  - operazioni da eseguire (es.: scansione o accesso diretto?)
  - ordine delle operazioni (es. join di tre relazioni; ordine?)
  - i dettagli del metodo (es.: quale metodo di join)
- Architetture parallele e distribuite aprono ulteriori gradi di libertà

Il processo di ottimizzazione

- Si costruisce un albero di decisione con le varie alternative ("**piani di esecuzione**")
- Si valuta il costo di ciascun piano (usando le statistiche o "profili" presenti nel catalogo)
- Si sceglie il piano di costo minore

- L'ottimizzatore trova di solito una "buona" soluzione, non necessariamente quella "ottima"

Un albero di decisione



Esempio

- Studenti(Matricola, Cognome, Nome, DataNascita, …)
- Esami(MatricolaStudente, CodiceCorso, Voto, Data)
- Corsi(Codice,Titolo,…)

- Trovare gli esami superati da Mario Rossi

```
SELECT Esami.*
FROM Studenti JOIN Esami ON Matricola=MatricolaStudente
WHERE Cognome LIKE 'Rossi' AND Nome Like 'Mario'
```

- Selezione su Studenti (per trovare Mario Rossi) e poi join con Esami

Progettazione fisica

- La fase finale del processo di progettazione di basi di dati
- input
  - lo schema logico e informazioni sul carico applicativo
- output
  - schema fisico, costituito dalle definizione delle relazioni con le relative strutture fisiche (e molti parametri, spesso legati allo specifico DBMS)

Strutture fisiche nei DBMS relazionali

- Struttura primaria:
  - disordinata (heap, "unclustered")
  - ordinata ("clustered"), anche su una pseudochiave
  - hash ("clustered"), anche su una pseudochiave, senza ordinamento
  - clustering di più relazioni (vedremo in un esercizio)
- Indici (densi/sparsi, semplici/composti):
  - ISAM (statico), di solito su struttura ordinata
  - B-tree (dinamico)
  - Indici hash (secondario, poco dinamico)

Strutture fisiche in alcuni DBMS

- Oracle:
  - struttura primaria
    - file heap
    - "hash cluster" (cioè struttura hash)
    - cluster (anche plurirelazionali) anche ordinati (con B-tree denso)
  - indici secondari di vario tipo (B-tree, bit-map, funzioni)
- DB2:
  - primaria: heap o ordinata con B-tree denso
  - indice sulla chiave primaria (automaticamente)
  - indici secondari B-tree densi
- SQL Server:
  - primaria: heap o ordinata con indice B-tree sparso
  - indici secondari B-tree densi

Strutture fisiche in alcuni DBMS, 2

- Ingres (molti anni fa):
  - file heap, hash, ISAM (ciascuno anche compresso)
  - indici secondari
- Informix (molti anni fa):
  - file heap
  - indici secondari (e primari [cluster] ma non mantenuti)

Definizione degli indici SQL

- Non è standard, ma presente in forma simile nei vari DBMS
  - `create` [`unique`] `index` *IndexName* `on` *TableName*(*AttributeList*)
  - `drop index` *IndexName*

- `unique` è un po' contraddittorio

Postgres, comandi su strutture fisiche

CREATE [UNIQUE] INDEX [name] ON table [USING method]
({ column | (expression) } [COLLATE collation] [opclass] [ASC | DESC] [NULLS { FIRST | LAST }] [, ...]) [WITH (storage_parameter = value [, ...])] [TABLESPACE tablespace] [WHERE predicate]

CLUSTER [VERBOSE] table_name [USING index_name]

VACUUM [({ FULL | FREEZE | VERBOSE | ANALYZE } [, ...])] [table [(column [, ...])]]

- Guardare la documentazione

Progettazione fisica nel modello relazionale

- La caratteristica comune dei DBMS relazionali è la disponibilità degli indici:
  - la progettazione fisica spesso coincide con la scelta degli indici (oltre ai parametri strettamente dipendenti dal DBMS)
- Le chiavi (primarie) delle relazioni sono di solito coinvolte in selezioni e join: molti sistemi prevedono (oppure suggeriscono) di definire indici sulle chiavi primarie
- Altri indici vengono definiti con riferimento ad altre selezioni o join "importanti"
- Se le prestazioni sono insoddisfacenti, si "tara" il sistema aggiungendo o eliminando indici
- È utile verificare se e come gli indici sono utilizzati con il comando SQL `show plan` oppure `explain`

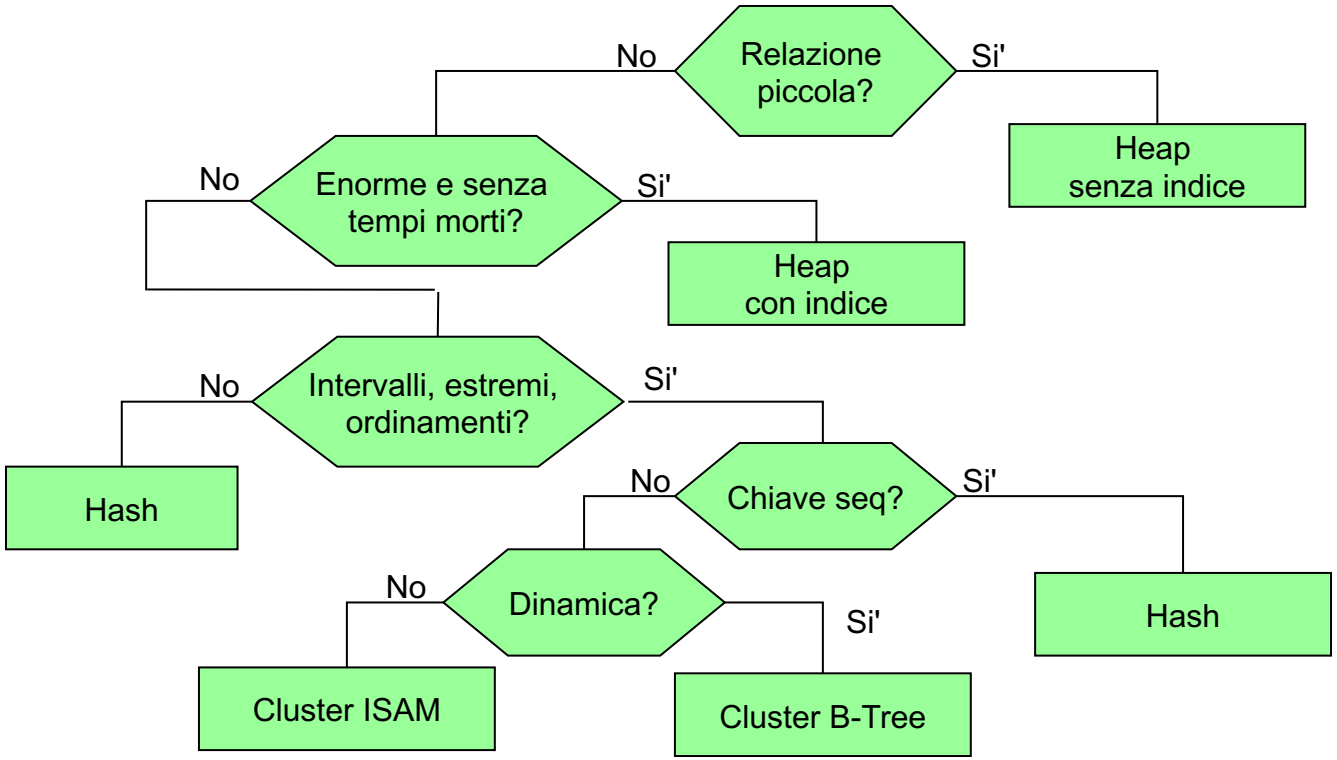
Progettazione fisica: euristiche suggerite da Informix

- Non creare indici su relazioni piccole (<200 ennuple)
- non creare indici su campi con pochi valori (se proprio servono, che siano primari)
- creare indici su campi con selezioni
- per i join: creare indici sulla relazione più grande

Scelta della struttura secondo Shasha

D. Shasha. Database Tuning: a principled approach. Prentice-Hall, 1992

D. Shasha, P. Bonnet Database Tuning: Principles, Experiments, and Troubleshooting Techniques Morgan Kaufmann 2002



Esempio reale con Postgres

```
set search_path to "20180328proveOttimizzazione";

drop table if exists r1;
create table r1 (    a numeric not null primary key,
                     b numeric,
                     c numeric);
drop table if exists r2;
create table r2 (    d numeric not null primary key,
                     e numeric,
                     f numeric);
drop table if exists r3;
create table r3 (    g numeric not null primary key,
                     h numeric,
                     i numeric);
alter table r1 add constraint fk1 foreign key (c) references r2 (d);
alter table r2 add constraint fk2 foreign key (f) references r3 (g);
```

Esempio, continua

```
create or replace function loadr3() returns void as $$
begin
    for i in 1..1000000 loop insert into r3
    values(i,floor(random()*20+1),floor(random()*1000)+1); end
    loop;
end;
$$ language plpgsql;
select loadr3();
…
create index i12 on r1(B);
create index i13 on r1(C);

vacuum analyze;
```

Esempio reale , segue

```
select * from r1 where B>=1 and B<=10;

select * from r1 where B>=1 and B<=2;

select * from r1 where C>=1 and C<=10;

select * from r1 where C>=1 and C<=2;

SELECT  A , E
FROM    R1, R3, R2
WHERE   C=D AND B<40 AND F=G AND A=7 AND I>2;

SELECT  A , E
FROM    R1, R3, R2
WHERE   C=D AND B<40 AND F=G AND I>2

...
```

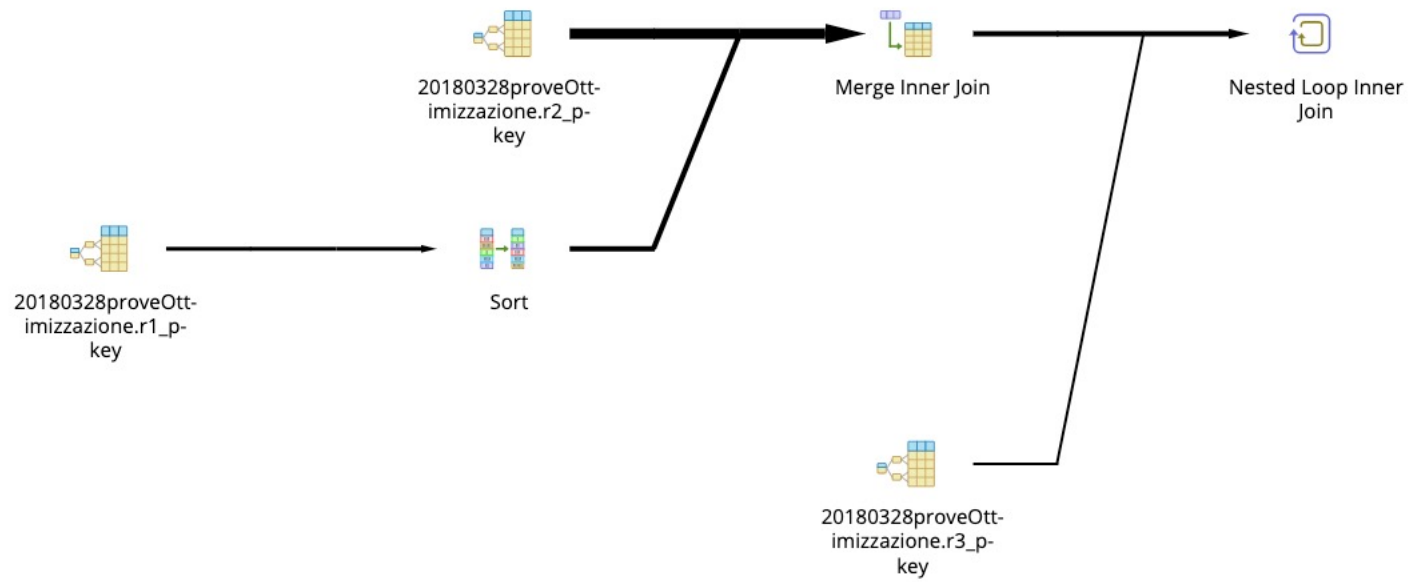
Fate i vostri test!

Explain

Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Query - postgres on paolo@PostgresSQL 11 *

No limit

postgres on paolo@PostgresSQL 11

```
1  SELECT  A , E
2  FROM    R1, R3, R2
3  WHERE   C=D AND B<40 AND F=G AND A=7 AND I>2;
4
5
```

Data Output | Explain | Messages | Notifications | Query History